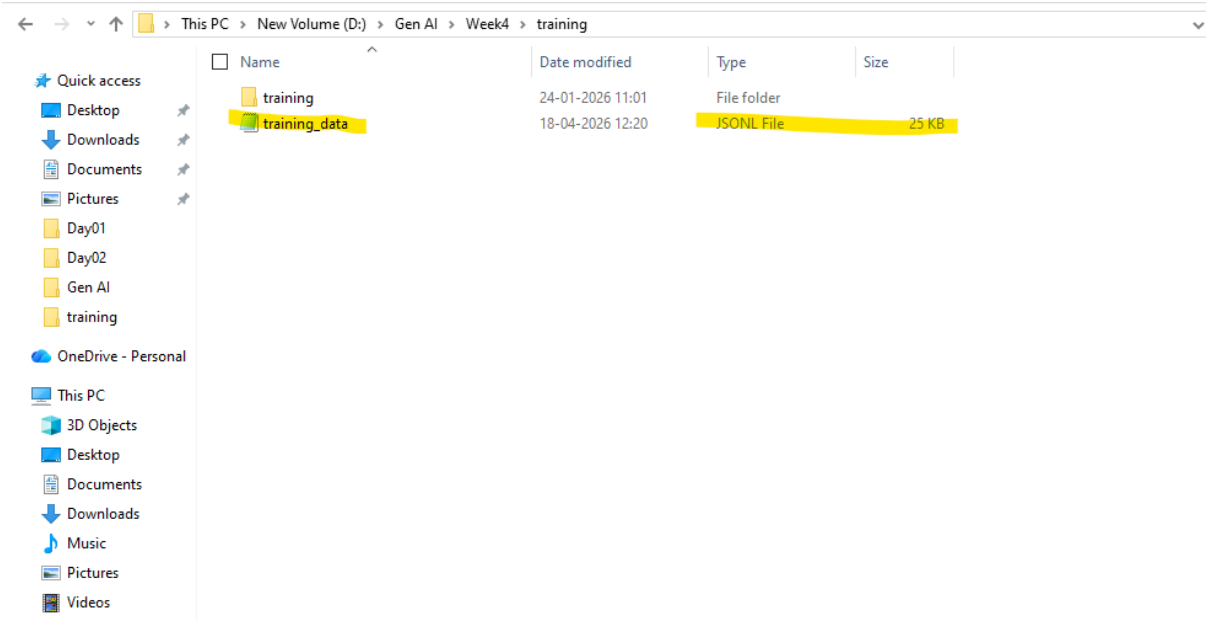
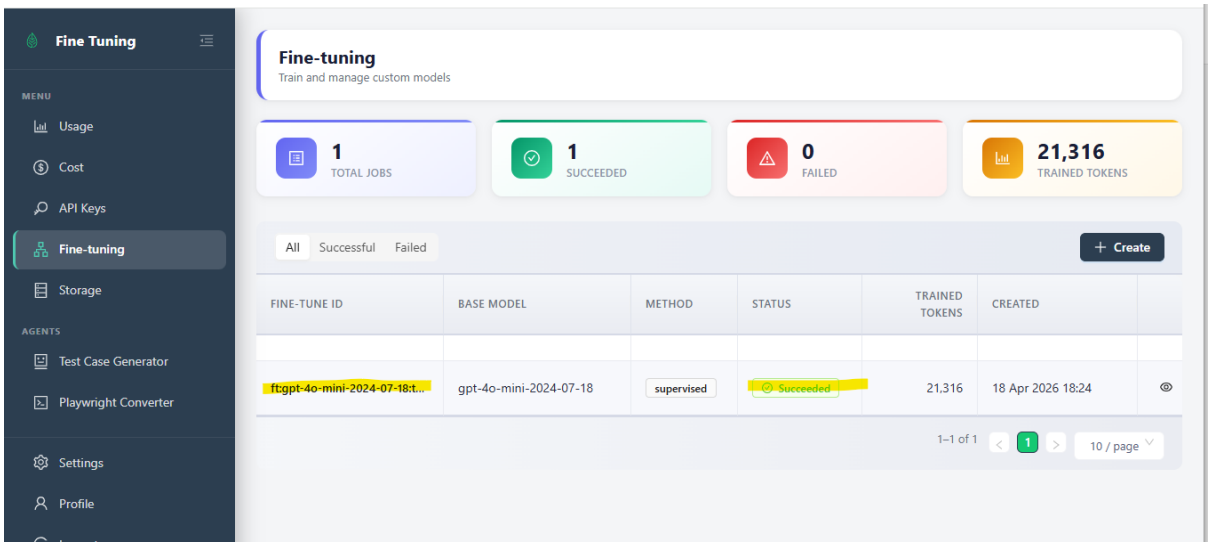
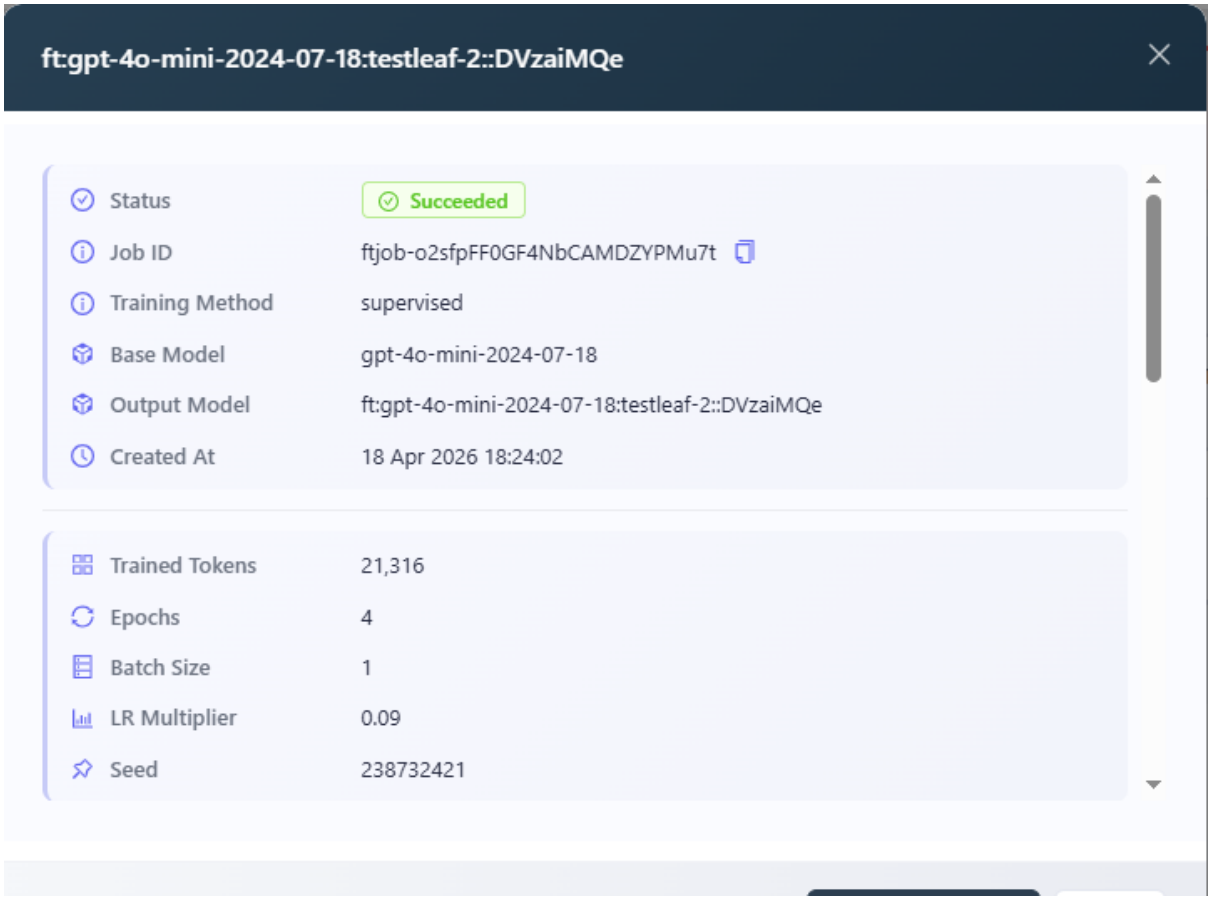
Creation of JSONL file:

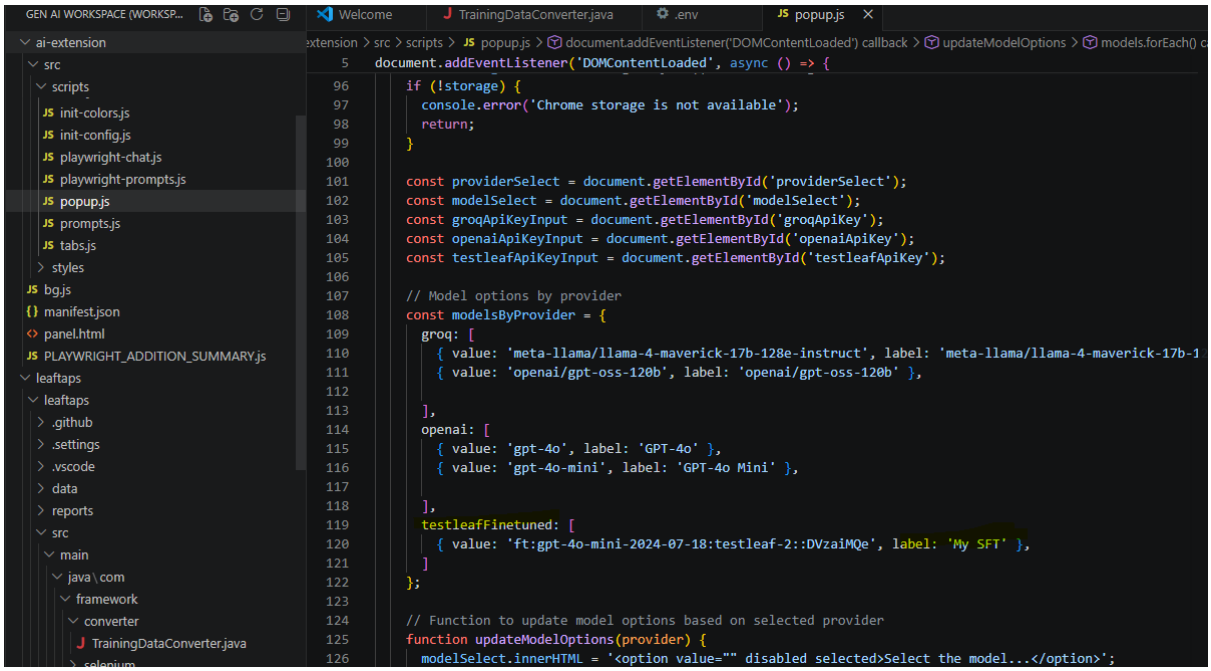


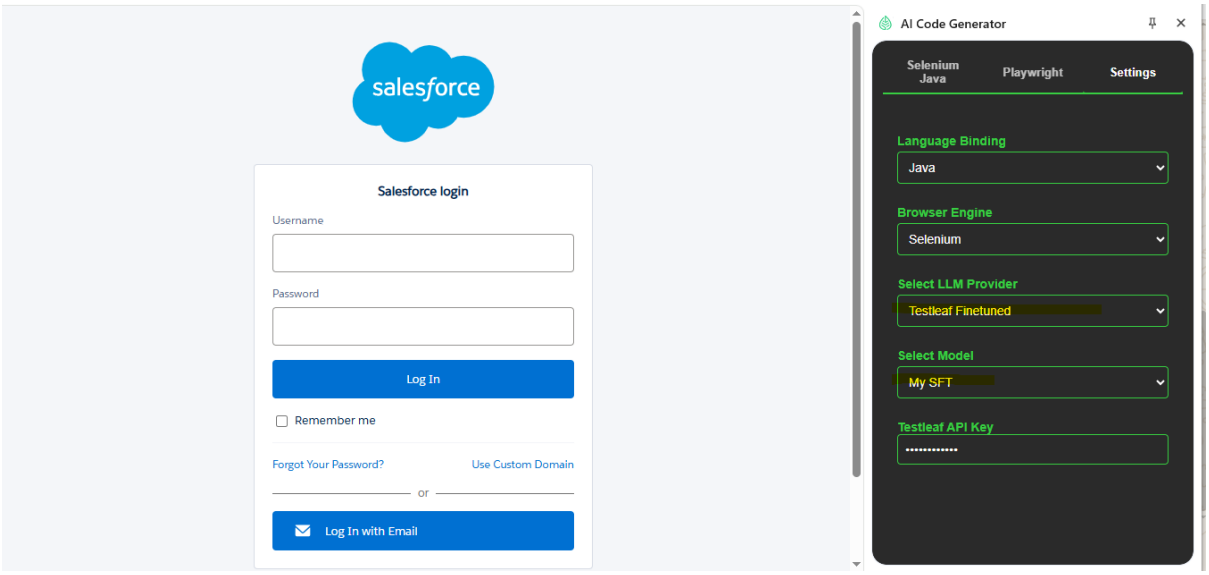
Creating Fine-tuned model:



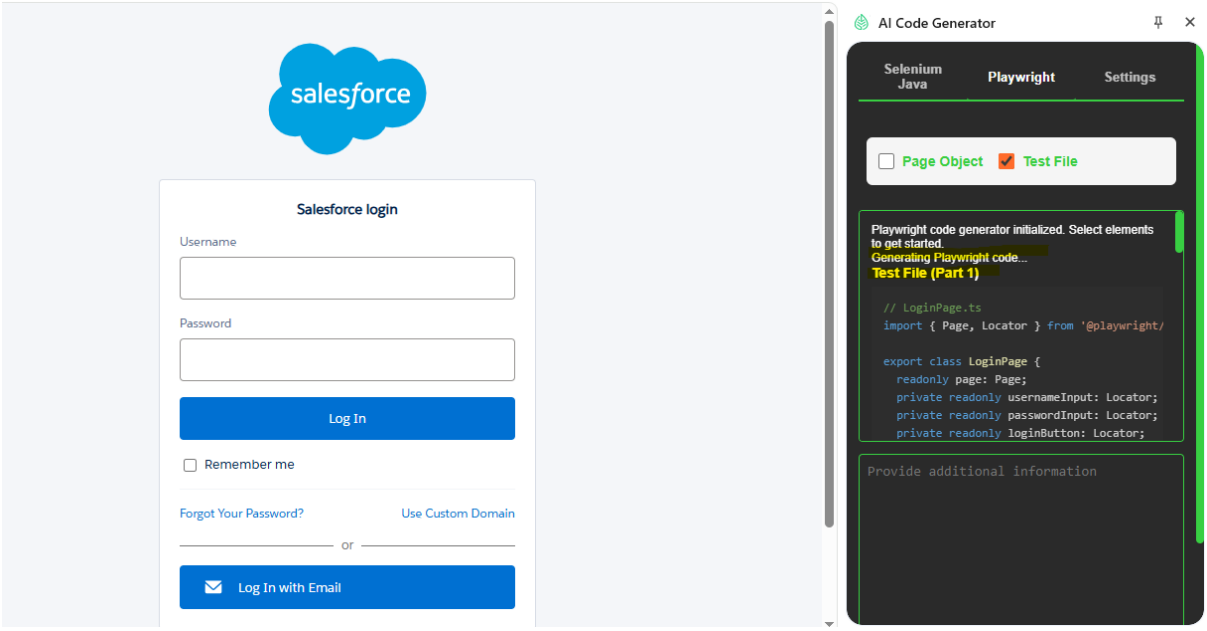


Adding the finetuned Model to the extension:





Generating code using the finetuned Model:



Code Generated:

Test File (Part 1)

```
// LoginPage.ts
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  private readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('input[type="submit"]#Login');
    this.errorMessage = page.locator('#chooser_error');
```

```
  }

  async goto() {
    await this.page.goto('chrome-extension://pckceenoololhcohnhnijkncjogalpdk/panel.html');
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async getErrorMessage(): Promise<string | null> {
    return await this.errorMessage.textContent();
  }

  async isErrorMessageVisible(): Promise<boolean> {
    return await this.errorMessage.isVisible();
  }
}
```

Test File (Part 2)

```
// login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test.describe('Login Feature', () => {
  let loginPage: LoginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('should successfully login with valid credentials', async () => {
    await loginPage.login('arjun.murthy@example.com', 'SecurePass@123');
    expect(await loginPage.isErrorMessageVisible()).toBeFalsy();
  });

  test('should display error with invalid credentials', async () => {
    await loginPage.login('arjun.murthy@example.com', 'wrongpassword');
    const errorMsg = await loginPage.getErrorMessage();
    expect(errorMsg).toContain('Invalid');
  });
});
```